



Marwadi
University
Marwadi Chandarana Group



MARWADI UNIVERSITY
FACULTY OF COMPUTER APPLICATIONS
PYTHON PROGRAMMING (05CS2104) – LAB MANUAL

Python Programming (05CS2104)

Lab Manual (2025-26)

Program: M.Sc. (Cyber Security & Cyber Law)

Semester: 1

Name of the Student: Jenil Makwana

Enrolment No.: 92600565009

INDEX

Lab	Program	Date	Remarks	Signature
1	Python Program to Print Hello World.	9-8-26		
2	Python program to do arithmetical operations	9-8-26		
3	Python program to swap two variables	9-8-26		
4	Python Program to Find the Factorial of a Number	9-8-26		
5	Python Program to Print the Fibonacci sequence	9-8-26		
6	Python program to print the elements of an array	11-8-26		
7	Python program to print the largest element and smallest element in an array	11-8-26		
8	Python program to add two matrix using array and function	11-8-26		
9	Python program to print the sum of all elements in an array	11-8-26		
10	Python Program to concatenate two strings	11-8-26		
11	Python Programs to perform various operations on Strings using functions	1-9-26		
12	Python Programs to demonstrate use of list and various functions of it	1-9-26		
13	Python Programs to demonstrate use of tuple and various functions of it	1-9-26		
14	Python Programs to demonstrate the use of dictionary and various functions of it	1-9-26		
15	Python Programs to demonstrate use of various arguments which can be passed to functions	1-9-26		
16	Python Program to demonstrate the use of class	1-9-26		
17	Python program to demonstrate the use of methods	1-9-26		
18	Python programs to demonstrate the use of various forms of inheritance	1-9-26		
19	Python programs to demonstrate use of modules	1-9-26		
20	Python programs to demonstrate the use of exception handling	1-9-26		
21	Python programs to demonstrate the use of file handling			
22	Python Programs to demonstrate the use of various modes of files.			
23	Python programs to demonstrate the use of regular expression			
24	Python programs to demonstrate the use of database using mysql			

25	Python Program to demonstrate the use of DataFrames in python			
26	Python Programs to demonstrate different ways of creating DataFrames			
27	Python Programs to create Pie chart			
28	Python Programs to create histogram			
29	Python Programs to create bar graph			
30	Python Programs to create Line graph			
31	Python Programs to work with Python Libraries useful for Cyber security. Program to generate hash values of a string using hashlib library.			
32	Python Program using socket — Simple Port Scanner			
33	Python Program using re — Detect Emails Using Regular Expressions			
34	Python Program using os Library — Check Files in a Directory			
35	Python program for whois lookup			
36	Python program for Ping Scanner shows ip up or down			

Enrollment/Roll No: 92600565009

Practical No. 1

Aim/ Objective: Python Program to Print Hello World.

Source Code:

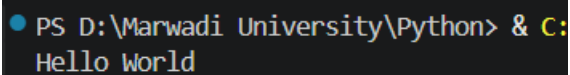
#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.1 Python Program to Print Hello World.

Print ("Hello World")

Output:



```
PS D:\Marwadi University\Python> & C:
Hello World
```

Conclusion: This program introduces the absolute foundation of Python syntax, demonstrating how to use the built-in print () function to display basic textual data directly to the console output.

Enrollment/Roll No: 92600565009

Practical No. 2

Aim/ Objective: Python program to do arithmetical operations

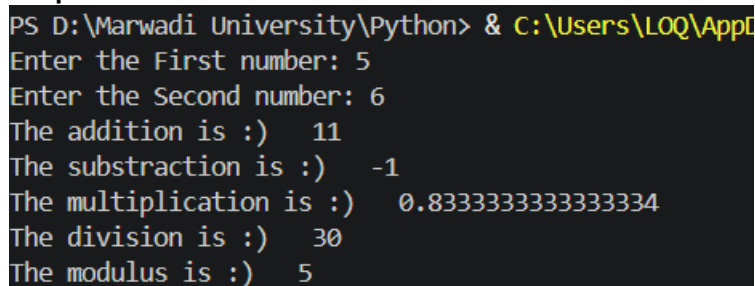
Source Code:

```
#Name: Jenil Makwana  
#Enrollment: 92600565009
```

#Q.2 Python program to do arithmetical operations

```
num1 = int(input("Enter the First number: "))  
num2 = int(input("Enter the Second number: "))  
  
print ("The addition is :) ",num1+num2)  
print ("The subtraction is :) ",num1-num2)  
print ("The multiplication is :) ",num1/num2)  
print ("The division is :) ",num1*num2)  
print ("The modulus is :) ",num1%num2)
```

Output:



```
PS D:\Marwadi University\Python> & C:\Users\LOQ\AppData\Local\Microsoft\WindowsApps\python.exe  
Enter the First number: 5  
Enter the Second number: 6  
The addition is :) 11  
The subtraction is :) -1  
The multiplication is :) 0.8333333333333334  
The division is :) 30  
The modulus is :) 5
```

Conclusion: This script shows how Python handles math using standard arithmetic operators (+, -, *, /) to calculate values and store numerical results inside basic variables.

Enrollment/Roll No: 92600565009

Practical No. 3

Aim/ Objective: Python program to swap two variables

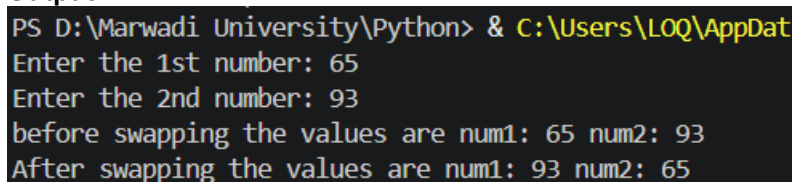
Source Code:

```
#Name: Jenil Makwana  
#Enrollment: 92600565009
```

#Q.3 Python program to Swap two variables

```
num1 = int(input("Enter the 1st number: "))  
num2 = int(input("Enter the 2nd number: "))  
  
print (f"before swapping the values are num1: {num1} num2: {num2}")  
  
num1,num2=num2,num1  
  
print (f"After swapping the values are num1: {num1} num2: {num2}")
```

Output:



```
PS D:\Marwadi University\Python> & C:\Users\LOQ\AppData  
Enter the 1st number: 65  
Enter the 2nd number: 93  
before swapping the values are num1: 65 num2: 93  
After swapping the values are num1: 93 num2: 65
```

Conclusion: This problem teaches logic and memory storage by showing why a tracking variable (temp) is needed to safely trade values between two data locations without overwriting them.

Enrollment/Roll No: 92600565009

Practical No. 4

Aim/ Objective: Python Program to Find the Factorial of a Number

Source Code:

```
#Name: Jenil Makwana  
#Enrollment: 92600565009
```

#Q.4 Python program to find factorial of number

```
num = int(input("Enter the number : "))  
  
fact=1  
for i in range (1,num+1):  
    fact *= i  
print ("The factorial of given number is : ",fact)
```

Output:

```
PS D:\Marwadi University\Python> & C:\Users\LOQ\AppData\Local\Microsoft\Windows\PowerShell\PowerShell.exe  
Enter the number : 5  
The factorial of given number is : 120
```

Conclusion: This code demonstrates a sequential loop calculation, showing how a single running product variable can repeatedly multiply growing numbers together to find a final mathematical total.

Enrollment/Roll No: 92600565009

Practical No. 5

Aim/ Objective: Python Program to Print the Fibonacci sequence

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.5 Python program to find fibonacci series

```
n=int(input("Enter the number: "))
```

```
a=0
```

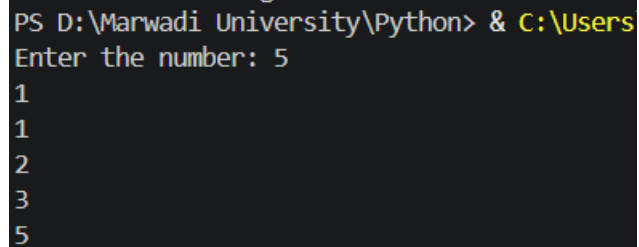
```
b=1
```

```
for i in range (n):
```

```
    a,b=b,a+b
```

```
    print(a)
```

Output:



```
PS D:\Marwadi University\Python> & C:\Users\jenil\...  
Enter the number: 5  
1  
1  
2  
3  
5
```

Conclusion: This exercise highlights variable updating and tracking, where two distinct numbers (a and b) pass changing values to each other dynamically to build a step-by-step sequence

Enrollment/Roll No: 92600565009

Practical No. 6

Aim/ Objective: Python program to print the elements of an array

Source Code:

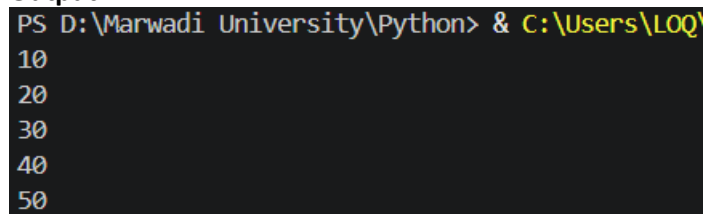
```
#Name: Jenil Makwana  
#Enrollment: 92600565009
```

#Q.6 Python program to print the elements of an array

```
arr = [10, 20, 30, 40, 50]
```

```
for i in arr:  
    print(i)
```

Output:



```
PS D:\Marwadi University\Python> & C:\Users\LOQ'  
10  
20  
30  
40  
50
```

Conclusion: This script teaches array navigation, showing how a basic iterative loop sequentially unpacks a list to isolate and inspect elements one item at a time from start to finish.

Enrollment/Roll No: 92600565009

Practical No. 7

Aim/ Objective: Python program to print the largest element and smallest element in an array

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.7 Python program to print the largest element and smallest element in an array

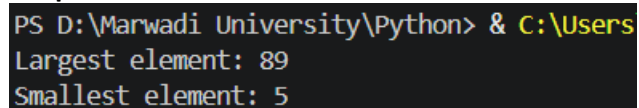
```
arr = [23, 5, 89, 45, 12, 7]
```

```
largest = arr[0]  
smallest = arr[0]
```

```
for i in arr:  
    if i > largest:  
        largest = i  
    if i < smallest:  
        smallest = i
```

```
print ("Largest element:", largest)  
print ("Smallest element:", smallest)
```

Output:



```
PS D:\Marwadi University\Python> & C:\Users  
Largest element: 89  
Smallest element: 5
```

Conclusion: This program teaches conditional data filtering, using simple if comparisons inside a scanning loop to evaluate list items and track the current extreme values.

Enrollment/Roll No: 92600565009

Practical No. 8

Aim/ Objective: Python program to add two matrix using array and function

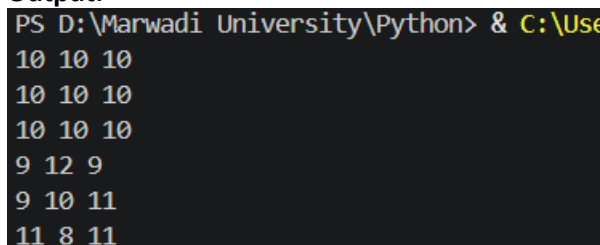
Source Code:

```
#Name: Jenil Makwana  
#Enrollment: 92600565009
```

```
#Q.8 Python program to add two matrix using array and function
```

```
#Using array  
a= [[1,2,3],[4,5,6],[7,8,9]]  
b= [[9,8,7],[6,5,4],[3,2,1]]  
  
for i in range (len(a)):  
    for j in range (len(a[0])):  
        print(a[i][j]+b[i][j], end=" ")  
    print ("")  
  
#Using Function  
def matrix_addition():  
    a= [[1,3,2],[5,4,6],[8,7,9]]  
    b= [[8,9,7],[4,6,5],[3,1,2]]  
  
    for i in range (len(a)):  
        for j in range (len(a[0])):  
            print(a[i][j]+b[i][j], end=" ")  
        print ("")  
  
matrix_addition()
```

Output:



```
PS D:\Marwadi University\Python> & C:\Use  
10 10 10  
10 10 10  
10 10 10  
9 12 9  
9 10 11  
11 8 11
```

Conclusion: This structure demonstrates multi-dimensional indexing, utilizing a nested loop layout (a loop inside a loop) to target corresponding row and column intersections across distinct data grids.

Enrollment/Roll No: 92600565009

Practical No. 9

Aim/ Objective: Python program to print the sum of all elements in an array

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.9 Python program to print the sum of all elements in an array

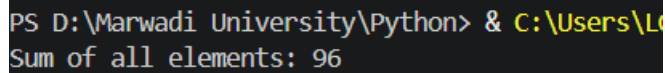
```
arr = [10, 2, 30, 4, 50]
```

```
sum = 0
```

```
for i in arr:  
    sum += i
```

```
print ("Sum of all elements:",sum)
```

Output:



```
PS D:\Marwadi University\Python> & C:\Users\Le...  
Sum of all elements: 96
```

Conclusion: This problem highlights iterative value aggregation, using a linear loop to visit list elements sequentially and add their values to a single running total tracking variable.



Enrollment/Roll No: 92600565009

Practical No. 10

Aim/ Objective: Python Program to concatenate two strings

Source Code:

```
#Name: Jenil Makwana
```

```
#Enrollment: 92600565009
```

```
#Q.10 Python Program to concatenate two strings
```

```
s1= "hello"
```

```
s2= "world"
```

```
print(s1+" "+s2)
```

Output:

```
PS D:\Marwadi University\Python> & C:\U  
hello world
```

Conclusion: This logic shows how Python handles text combination, using the addition sign (+) as a simple text joining tool to merge individual pieces of string data together.

Enrollment/Roll No: 92600565009

Practical No. 11

Aim/ Objective: Python Program to demonstrate use of list and various functions of it.

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.11 Python Program to demonstrate use of list and various functions of it

```
print("Name: Jenil Makwana\n Enrollment: 92600565009")
```

```
numbers = [10, 20, 30, 40, 50]
```

```
print("The list is: ", numbers)
```

```
numbers.append(60)
```

```
print("After append: ", numbers)
```

```
numbers.insert(2, 25)
```

```
print("After insert: ", numbers)
```

```
numbers.remove(40)
```

```
print("After remove: ", numbers)
```

```
numbers.sort()
```

```
print("After sort: ", numbers)
```

```
numbers.reverse()
```

```
print("After reverse: ", numbers)
```

```
print("The length of list is: ", len(numbers))
```

Output:

```
Name: Jenil Makwana
Enrollment: 92600565009
The list is: [10, 20, 30, 40, 50]
After append: [10, 20, 30, 40, 50, 60]
After insert: [10, 20, 25, 30, 40, 50, 60]
After remove: [10, 20, 25, 30, 50, 60]
After sort: [10, 20, 25, 30, 50, 60]
After reverse: [60, 50, 30, 25, 20, 10]
The length of list is: 6
```

Conclusion: This logic demonstrates how Python handles lists, which are mutable data structures, using built-in functions like `append()`, `insert()`, `remove()`, `sort()`, and `reverse()` to actively modify and organize the stored elements.

Enrollment/Roll No: 92600565009

Practical No. 12

Aim/Objective: Python Program to demonstrate use of tuple and various functions of it.

Source Code:

#Q.12 Python Program to demonstrate use of list and various functions of it

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.12 Python Program to demonstrate use of tuple and various functions of it

```
print("Name: Jenil Makwana\n Enrollment: 92600565009")
```

```
numbers = (10, 20, 30, 20, 40, 50)
```

```
print("The tuple is: ", numbers)
```

```
print("The length of tuple is: ", len(numbers))
```

```
print("The count of 20 is: ", numbers.count(20))
```

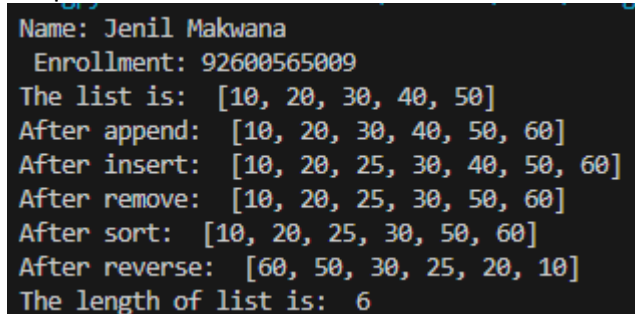
```
print("The index of 30 is: ", numbers.index(30))
```

```
print("The maximum value is: ", max(numbers))
```

```
print("The minimum value is: ", min(numbers))
```

```
print("The sum of tuple is: ", sum(numbers))
```

Output:



```
Name: Jenil Makwana
 Enrollment: 92600565009
The list is: [10, 20, 30, 40, 50]
After append: [10, 20, 30, 40, 50, 60]
After insert: [10, 20, 25, 30, 40, 50, 60]
After remove: [10, 20, 25, 30, 50, 60]
After sort: [10, 20, 25, 30, 50, 60]
After reverse: [60, 50, 30, 25, 20, 10]
The length of list is: 6
```

Conclusion: This logic shows how Python interacts with tuples. Since tuples are immutable, the program relies on read-only functions like `count()`, `index()`, `max()`, and `min()` to extract information and perform calculations.

Enrollment/Roll No: 92600565009

Practical No. 13

Aim/ Objective: Python Program to demonstrate the use of dictionary and various functions of it

Source Code:

#Q.13 Python Program to demonstrate the use of dictionary and various functions of it

#Name: Jenil Makwana

#Enrollment: 92600565009

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
student = {  
    "Name": "Jenil",  
    "Age": 22,  
    "Course": "MScCybersecurity"  
}
```

```
print("The dictionary is: ", student)
```

```
print("The keys are: ", student.keys())
```

```
print("The values are: ", student.values())
```

```
print("The items are: ", student.items())
```

```
student["City"] = "Rajkot"  
print("After adding City: ", student)
```

```
student.pop("Age")  
print("After removing Age: ", student)
```

```
print("The length of dictionary is: ", len(student))
```

Output:

```
Name: Jenil Makwana  
Enrollment: 92600565009  
The dictionary is: {'Name': 'Jenil', 'Age': 22, 'Course': 'MSc Cybersecurity'}  
The keys are: dict_keys(['Name', 'Age', 'Course'])  
The values are: dict_values(['Jenil', 22, 'MSc Cybersecurity'])  
The items are: dict_items([('Name', 'Jenil'), ('Age', 22), ('Course', 'MSc Cybersecurity')])  
After adding City: {'Name': 'Jenil', 'Age': 22, 'Course': 'MSc Cybersecurity', 'City': 'Rajkot'}  
After removing Age: {'Name': 'Jenil', 'Course': 'MSc Cybersecurity', 'City': 'Rajkot'}  
The length of dictionary is: 3
```

Conclusion: This program shows dictionary manipulation in Python, including accessing keys, values, and items, as well as adding and popping specific key-value pairs.

Enrollment/Roll No: 92600565009

Practical No. 14

Aim/Objective: Python Program to create a function

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.14 Python Program to create a function

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
def addition(num1, num2):  
    result = num1 + num2  
    print("The addition is: ", result)
```

```
num1 = int(input("Enter the First number: "))
```

```
num2 = int(input("Enter the Second number: "))
```

```
addition(num1, num2)
```

Output:

```
Name: Jenil Makwana  
Enrollment: 92600565009  
Enter the First number: 56  
Enter the Second number: 26  
The addition is: 82
```

Conclusion: This logic covers the basics of modular programming in Python by defining a custom addition() function, taking dynamic user input, and performing the arithmetic block independently from the main execution flow.

Enrollment/Roll No: 92600565009

Practical No. 15

Aim/Objective: Python Program to demonstrate the use of various arguments which can be passed to functions

Source Code:

```
#Name: Jenil Makwana  
#Enrollment: 92600565009
```

#Q.15 Python Program to demonstrate the use of various arguments which can be passed to functions

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
def student(name, age=21):  
    print("Student Name is: ", name)  
    print("Student Age is: ", age)
```

```
print("Using positional arguments:")  
student("Jenil", 22)
```

```
print("Using keyword arguments:")  
student(age=22, name="Rahul")
```

```
print("Using default arguments:")  
student("Karan")
```

Output:

```
Name: Jenil Makwana  
Enrollment: 92600565009  
Using positional arguments:  
Student Name is: Jenil  
Student Age is: 22  
Using keyword arguments:  
Student Name is: Rahul  
Student Age is: 22  
Using default arguments:  
Student Name is: Karan  
Student Age is: 21
```

Conclusion: This logic demonstrates the flexibility of Python function parameters by successfully parsing data through standard positional arguments, explicitly mapped keyword arguments, and a fallback default argument when the user omits a parameter.

Enrollment/Roll No: 92600565009

Practical No. 16

Aim/Objective: Python Program to demonstrate the use of class

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.16 Python Program to demonstrate the use of class

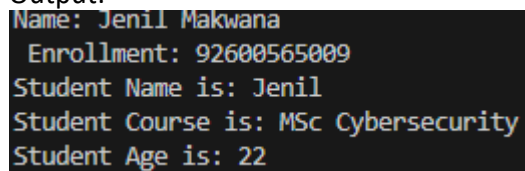
```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
class Student:  
def display(self):  
print("Student Name is: Jenil")  
print("Student Course is: MSc Cybersecurity")  
print("Student Age is: 22")
```

```
student1= Student()
```

```
student1.display()
```

Output:



```
Name: Jenil Makwana  
Enrollment: 92600565009  
Student Name is: Jenil  
Student Course is: MSc Cybersecurity  
Student Age is: 22
```

Conclusion: This logic introduces Object-Oriented Programming (OOP) in Python by encapsulating student details inside a Student class and demonstrating how to instantiate a discrete object to call the class's internal behavior.

Enrollment/Roll No: 92600565009

Practical No. 17

Aim/Objective: Python Program to demonstrate the concept of inner class

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.17 Python Program to demonstrate the concept of inner class

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
class Student:
```

```
def display(self):
```

```
print("This is the Student class")
```

```
class Address:
```

```
def display_address(self):
```

```
    print("City: Rajkot")
```

```
    print("Country: India")
```

```
student1 = Student()
```

```
student1.display()
```

```
address1 = student1.Address()
```

```
address1.display_address()
```

Output:

```
Name: Jenil Makwana
Enrollment: 92600565009
This is the Student class
City: Rajkot
Country: India
```

Conclusion: This logic illustrates nesting in Object-Oriented Programming by placing an Address class completely inside a Student class, showing how the inner class is dependent on the outer class namespace for instantiation.

Enrollment/Roll No: 92600565009

Practical No. 18

Aim/Objective: Python Program to demonstrate the use of methods

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.18 Python Program to demonstrate the use of methods

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
class Student:
```

```
def display(self):
```

```
print("Student Name is: Jenil")
```

```
print("Student Course is: MSc Cybersecurity")
```

```
student1 = Student()
```

```
student1.display()
```

Output:

```
Name: Jenil Makwana
Enrollment: 92600565009
Student Name is: Jenil
Student Course is: MSc Cybersecurity
```

Conclusion: This logic serves as a straightforward example of binding a function (method) to a class object, using the self parameter to represent the current instance of the class that is executing the code.

Enrollment/Roll No: 92600565009

Practical No. 19

Aim/Objective: Python Program to demonstrate various types of methods

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.19 Python Program to demonstrate various types of methods

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
class Student:
```

```
def instance_method(self):
```

```
    print("This is an instance method")
```

```
@classmethod
```

```
def class_method(cls):
```

```
    print("This is a class method")
```

```
@staticmethod
```

```
def static_method():
```

```
    print("This is a static method")
```

```
student1= Student()
```

```
student1.instance_method()
```

```
Student.class_method()
```

```
Student.static_method()
```

Output:

```
Name: Jenil Makwana
Enrollment: 92600565009
This is an instance method
This is a class method
This is a static method
```

Conclusion: This logic breaks down the three distinct method scopes in Python OOP using decorators, contrasting an instance method, a @classmethod, and a @staticmethod.

Enrollment/Roll No: 92600565009

Practical No. 20(A)

Aim/Objective: Python Program to show method overloading by adding two numbers and three numbers using the same method name

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.20(a) Python Program to show method overloading by adding two numbers and three numbers using the same method name

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
class Addition:
```

```
def add(self, num1, num2, num3=0):
```

```
    print("The addition is: ", num1 + num2 + num3)
```

```
addition = Addition()
```

```
addition.add(10, 20)
```

```
addition.add(10, 20, 30)
```

Output:

```
Name: Jenil Makwana
Enrollment: 92600565009
The addition is: 30
The addition is: 60
```

Conclusion: This logic replicates method overloading by cleverly utilizing a default argument, allowing the exact same add() method to handle varying numbers of inputs without throwing an error.

Enrollment/Roll No: 92600565009

Practical No. 20(B)

Aim/Objective: Python Program to show method overriding using a parent class Animal and a child class Dog

Source Code:

#Name: Jenil Makwana

#Enrollment: 92600565009

#Q.20(b) Python Program to show method overriding using a parent class Animal and a child class Dog

```
print("Name: Jenil Makwana \n Enrollment: 92600565009")
```

```
class Animal:  
    def sound(self):  
        print("Animal makes a sound")
```

```
class Dog(Animal):  
    def sound(self):  
        print("Dog barks")
```

```
animal = Animal()  
animal.sound()
```

```
dog = Dog()  
dog.sound()
```

Output:

```
Name: Jenil Makwana  
Enrollment: 92600565009  
Animal makes a sound  
Dog barks
```

Conclusion: This logic demonstrates inheritance and polymorphism via method overriding, bro. By defining a sound() method inside the Dog child class, the program allows the child class to completely replace the parent's default behavior with its own specific output.